Fine-Tuning and Serving Gemma 4 31B on Google Cloud TPU: A Technical Comparison with GPU Baselines

LoRA SFT, Checkpoint Conversion, and vLLM Inference
on TPU v5p and v6e (Trillium)

h2loop.ai Engineering

May 2026

Abstract

We present a comprehensive technical account of fine-tuning Gemma 4 31B using LoRA on Google Cloud TPU v5p-8, and serving the fine-tuned model for inference on TPU v6e-8 (Trillium). We document the specific code-level adaptations required to port the GPU training recipe (PyTorch + HuggingFace TRL + FSDP) to the JAX + Tunix/Qwix stack, including mesh configuration, LoRA module naming differences, sharding annotation fixes, gradient checkpointing, the data pipeline, and a custom Orbax-to-safetensors checkpoint merging procedure. For inference, we cover the vLLM-TPU Docker setup required to serve Gemma 4 on v6e-8 and explain the observed latency and throughput characteristics. Across both workloads we compare performance and cost against a 2×H100 GPU baseline running identical hyperparameters. The TPU completes training 1.61× faster at 2.12× lower cost. Inference throughput is within 3% on both platforms, but TPU delivers 2× lower time-to-first-token (235 ms vs 475 ms). End-to-end, TPU is 1.82× cheaper for a representative train-plus-serve workload.

Contents

1 Introduction

Large language model fine-tuning is increasingly bottlenecked not by data availability but by compute cost and iteration speed. Google Cloud TPUs, while natively supported for training, require non-trivial porting effort compared to the mature GPU ecosystem (PyTorch, HuggingFace, FSDP). This report documents our end-to-end experience running Gemma 4 31B LoRA supervised fine-tuning on TPU v5p-8 and vLLM inference on TPU v6e-8 (Trillium), and compares both to an existing GPU baseline on 2×H100 80GB (GCP `a3-highgpu-2g`).

The task is Verilog code generation: we fine-tune on the CodeV-R1 dataset [1] (10K samples of natural-language-to-Verilog pairs) and evaluate on NVlabs/verilog-eval [2] (156 spec-to-RTL problems, pass@1/pass@5 with iverilog simulation).

The paper is structured as a walkthrough of the engineering changes required to make TPU work, followed by quantitative results and cost analysis.

2 Hardware and Infrastructure

2.1 Training Hardware

Table 1: Training hardware configurations.

Spec	**TPU v5p-8**	**GPU a3-highgpu-2g**
Accelerator	4× TPU v5p chips	2× NVIDIA H100 80GB HBM3
HBM per chip	102.8 GB	80 GB
Total HBM	411.2 GB	160 GB
Host RAM	˜355 GB	˜468 GB
Interconnect	ICI (inter-chip)	NVLink
GCP on-demand rate	$16.80/hr	$22.12/hr

2.2 Inference Hardware

Table 2: Inference hardware configurations.

Spec	**TPU v6e-8 (Trillium)**	**GPU a3-highgpu-2g**
Accelerator	8× TPU v6e chips	2× NVIDIA H100 80GB HBM3
HBM per chip	31.25 GB	80 GB
Total HBM	250 GB	160 GB
Tensor parallelism	tp=8	tp=2
GCP on-demand rate	$21.52/hr	$22.12/hr

2.3 TPU VM Environment

Both TPU generations (v5p and v6e) ship with Python 3.10, but the Tunix framework requires Python 3.11+. The first setup step is therefore installing Python 3.11 via the `deadsnakes` PPA before anything else. The JAX TPU wheel is installed from a separate Google-hosted index:

```
pip install "jax[tpu]" -f
    https://storage.googleapis.com/jax-releases/libtpu_releases.html
```

A critical operational note: the TPU VM's boot disk (97 GB) cannot hold the 62.5 GB Gemma 4 31B model download alongside the OS. Tunix stages the model in `/tmp/models` before loading into HBM. On the v5p-8, the host has 355 GB of RAM-backed `/dev/shm` (tmpfs), which we redirect the staging to via `TMPDIR=/dev/shm` at launch time. Without this, the model download fills the boot disk and crashes mid-load.

For the v6e-8 (Trillium) inference instance, we found that only the `v2-alpha-tpuv6e` runtime works correctly—the generic `tpu-ubuntu2204-base` runtime fails at TPU topology enumeration. Zone availability for v6e-8 was constrained; we used `asia-northeast1-b` as the only available zone at time of testing.

3 TPU Training Recipe

3.1 Framework Differences: PyTorch vs JAX

The GPU baseline uses PyTorch + HuggingFace TRL (`SFTTrainer`) with FSDP for model parallelism. The TPU recipe uses JAX + Tunix (`PeftTrainer`) with Qwix for LoRA injection. These are fundamentally different execution models:

- **GPU**: Eager PyTorch with FSDP sharding. Each GPU sees its shard of the model; gradients are reduced across ranks by FSDP's all-reduce.
- **TPU**: XLA-compiled JAX with an explicit device mesh. The model is sharded via `jax.sharding.PartitionSpec` annotations; XLA generates the communication collectives automatically. The first training step incurs a 3–8 minute XLA compilation cost; subsequent steps execute the compiled graph.

The key implication is that TPU training is less tolerant of dynamic shapes and control flow that exits traced functions. Every shape that varies across steps triggers a recompile.

3.2 Device Mesh Configuration

JAX requires an explicit 2D device mesh specifying how chips are arranged across FSDP and tensor-parallel (TP) dimensions. For the v5p-8 (4 chips):

```
# v5p-8 has 4 chips; tp=8 breaks on Gemma4 heads not divisible by 8
MESH_COUNTS = (1, 4)   # fsdp=1, tp=4
mesh = jax.make_mesh(
    MESH_COUNTS, ("fsdp", "tp"),
    axis_types=(jax.sharding.AxisType.Auto,) * 2,
)
```

The `tp=8` configuration was attempted but fails for Gemma 4 31B because the model's attention head count (32 heads, 16 KV heads) is not divisible by 8 for all projection shapes. `tp=4` works correctly. On a hypothetical 16-chip (v5p-16) configuration, the mesh would be `(2, 8)`.

3.3 LoRA Module Naming: Tunix vs HuggingFace

The GPU recipe targets LoRA modules via a HuggingFace PEFT regex:

```
^(?!.*vision).*\.(q_proj|k_proj|v_proj|o_proj|gate_proj|up_proj|down_proj)$
```

The HuggingFace model has separate `q_proj`, `k_proj`, `v_proj`, and `o_proj` Linear layers. The vision-tower exclusion regex (`^(?!.*vision)`) is required because the HF Gemma 4 checkpoint is multimodal and the vision tower contains similarly-named projections.

Tunix's JAX port of Gemma 4 uses different module names:

Table 3: LoRA target module name mapping between HF PyTorch and Tunix JAX.

HF PyTorch (GPU)	Tunix JAX (TPU)	Notes
q_proj	q_einsum	Renamed; same weights
k_proj + v_proj	kv_einsum	**Fused** into a single tensor with dim-1 = {K,V}
o_proj	attn_vec_einsum	Renamed
gate_proj	gate_proj	Same
up_proj	up_proj	Same
down_proj	down_proj	Same

The `kv_einsum` fusion is the most significant difference: the Tunix model stores both K and V projections in a single weight tensor of shape `(in, 2, n_kv_heads, head_dim)`, while HuggingFace stores them as separate `(n_kv_heads*head_dim, in)` matrices. This requires special handling during checkpoint merging (see Section 4).

There is also no vision tower in the JAX port, so the exclusion regex is unnecessary:

```
_LORA_MODULES = (
    ".*q_einsum|.*kv_einsum|.*attn_vec_einsum"
    "|.*gate_proj|.*down_proj|.*up_proj"
)
```

3.4 LoRA Sharding Annotation Fixes

Qwix injects LoRA parameters by tracing the model and inserting `lora_a`/`lora_b` tensors. However, it inherits the sharding annotations (`PartitionSpec`) from the original weight tensor—which has a different rank than the LoRA factors. For example, the `kv_einsum` base weight has shape `(in, 2, n_kv_heads, head_dim)` with a 4-element PartitionSpec, but the LoRA `lora_a` has shape `(in, rank)` (rank 2). When the optimizer tries to shard optimizer states according to this mismatched spec, JAX raises an error.

We fix this with a post-injection pass that resets any PartitionSpec whose rank doesn't match the actual tensor rank to a fully-replicated spec:

```
for _, module in nnx.iter_modules(lora_model):
    for attr_name in list(vars(module)):
        v = getattr(module, attr_name, None)
        if not isinstance(v, nnx.Variable):
            continue
        shd_val = getattr(v, 'sharding', None)
        if isinstance(shd_val, tuple) and len(shd_val) != v[...].ndim:
            v.set_metadata('out_sharding', (None,) * v[...].ndim)
```

A second pass checks divisibility: if a tensor dimension is not divisible by the mesh size for the assigned axis, we replicate that axis to avoid uneven sharding:

```
for i, axis_name in enumerate(shd_val):
    if axis_name is not None:
        mesh_size = MESH_COUNTS[["fsdp","tp"].index(axis_name)]
        if v[...].shape[i] % mesh_size != 0:
            safe[i] = None   # replicate this axis
```

The same fix is applied to the optimizer state via a monkey-patched `_shard_optimizer` that filters the partition specs through the same divisibility check before applying `jax.lax.with_sharding_constraint`.

3.5 Gradient Checkpointing

On TPU, gradient checkpointing (called *rematerialization* in JAX) is applied at the decoder layer granularity. Tunix exposes this via `nnx.remat`:

```
from tunix.models.gemma4 import model as _gemma4_mod
_gemma4_mod.DecoderLayer.__call__ = nnx.remat(
    _gemma4_mod.DecoderLayer.__call__
)
```

This patches the unbound method directly, so all decoder layer instances use the rematerialized forward pass. Without this, the 31B model's activation memory during backward would exceed the v5p-8's 411 GB HBM at the batch sizes used. The GPU equivalent is HuggingFace's `activation_checkpointing=True` in the FSDP config.

3.6 XLA Compiler Flags

Two environment variables are set before any JAX import:

```
export TMPDIR=/dev/shm           # route model staging to RAM-backed
    tmpfs
export XLA_FLAGS="--xla_llvm_disable_expensive_passes=true"
```

The `xla_llvm_disable_expensive_passes` flag skips several LLVM optimization passes that are particularly slow for large Transformer graphs. This trades a small amount of runtime throughput (˜2–5%) for significantly faster XLA compilation (from ˜8 min to ˜3 min for the first step).

3.7 Optimizer: Optax AdamW

The GPU recipe uses PyTorch's `adamw_torch` optimizer via HuggingFace Trainer. On TPU, JAX has no built-in optimizer; instead we use **Optax**, Google's composable gradient transformation library for JAX.

The schedule and optimizer are constructed as:

```
schedule = optax.warmup_cosine_decay_schedule(
    init_value=0.0,
    peak_value=1e-4,
    warmup_steps=100,
    decay_steps=MAX_STEPS,
    end_value=0.0,
)
optimizer = optax.adamw(learning_rate=schedule, weight_decay=0.001)
```

This mirrors the GPU recipe exactly: linear warmup from 0 to peak LR over 100 steps, followed by cosine decay to 0 over the remaining steps. The key differences from PyTorch AdamW are:

- **Composability**: Optax builds the optimizer as a chain of stateless gradient transformations (`scale_by_adam`, `add_decayed_weights`, `scale_by_schedule`). Each transformation has its own state pytree, which Orbax checkpoints independently.
- **No grad clipping by default**: PyTorch's HuggingFace Trainer applies `max_grad_norm=1.0` automatically. On TPU we omit this (matching the measured gradient norms, which stay stable without clipping), though `optax.clip_by_global_norm(1.0)` can be prepended to the chain if needed.
- **Schedule is a pure function**: The LR at any step is computed by calling `schedule(step)` — useful for logging and the warmup-clamp we apply when running short subsample runs.

- **Optimizer state sharding**: The Optax state (Adam moments) is sharded across the same device mesh as the model weights via `_shard_optimizer`, requiring the divisibility fix described in Section 3.4.

3.8 Data Pipeline: Grain vs HuggingFace DataLoader

The GPU recipe uses HuggingFace's `DataCollatorForCompletionOnlyLM` with `SFTTrainer`. The TPU recipe uses Google's `grain` library (a deterministic, shardable data loader). Key differences:

1. **Loss masking**: HuggingFace uses the {% generation %} chat template tag to identify assistant-turn tokens. Tunix's tokenizer adapter doesn't emit these markers, so we implement the mask by tokenizing user and model turns separately and computing offsets:

```
for msg in messages:
    header_ids = _enc_clean(f"<start_of_turn>{role}\n")
    body_ids   = _enc_clean(f"{content}<end_of_turn>\n")
    if role == "model":
        mask[header_end:body_end] = True    # only model turns
            contribute to loss
```

2. **System prompt handling**: The GPU recipe includes a system prompt in the HuggingFace chat template, but the Gemma template silently drops system messages—only `user` and `assistant` roles are rendered. The TPU recipe mirrors this exactly by emitting only user/model turns.
3. **Reasoning strip**: The CodeV-R1 dataset contains chain-of-thought reasoning in `<think>...</think>` blocks. We strip these and keep only the `<answer>` block (or the Verilog fence directly), reducing sequence lengths and focusing training on the final code output.
4. **Drop vs truncate**: Both recipes drop sequences longer than `max_seq_len` rather than truncating, to avoid training on incomplete Verilog modules. With `max_seq_len=3072`, 99.6% of the 10K-sample subset is retained (37 dropped as too long, 4 empty).

3.9 Checkpointing

The TPU recipe uses Orbax (`orbax-checkpoint`) with a GCS path for checkpoint storage. Checkpoints are saved every 100 optimizer steps, retaining the 3 most recent (matching the GPU recipe's `save_steps=500, save_total_limit=3` with a finer cadence for TPU's longer run). Checkpoints survive spot preemption because they are written directly to GCS rather than local disk.

4 Checkpoint Conversion: Orbax to Safetensors

After training, inference requires merging the LoRA adapters back into the base weights and saving a merged safetensors model. The GPU recipe uses HuggingFace PEFT's built-in `merge_and_unload()`. The TPU recipe requires a custom `orbax_to_peft.py` due to:

1. Orbax checkpoints store the full model tree including LoRA wrappers in a JAX-specific format (not safetensors).
2. The Tunix JAX model uses different weight names and tensor layouts than HuggingFace safetensors (which is what Tunix's sampler ultimately loads from for inference).
3. The `kv_einsum` fusion means one LoRA module maps to *two* safetensors keys.

4.1 Merge Process

The merging steps are:

1. Load the base model (from GCS safetensors) into JAX with the mesh.
2. Inject LoRA via Qwix (same structure as training) using dummy inputs.
3. Restore the Orbax checkpoint into the LoRA model (LoRA params only).
4. Iterate over all `nnx.LoRAParam` tensors, collect `lora_a`/`lora_b` pairs per module.
5. Load the raw base safetensors weights into a mutable NumPy dict.
6. Apply the LoRA delta: $W_{\text{merged}} = W_{\text{base}} + \frac{\alpha}{r} \cdot AB$
7. Save the merged dict as a single `model.safetensors` file.

4.2 Tensor Shape Mapping

The delta computation must account for different tensor layouts between JAX (Tunix) and HuggingFace:

Table 4: LoRA delta computation per module type.

Module	JAX shapes	Delta & layout
`q_einsum`	A: `(in, r)`, B: `(r, heads, head_dim)`	$\Delta = (AB)^T \rightarrow$ `(out, in)`
`kv_einsum`	A: `(in, r)`, B: `(r, 2, kv_heads, hd)`	Split B on dim-1; K=$B[:, 0, :]$, V=$B[:, 1, :]$; $\Delta = (AB)^T$ for each
`attn_vec_einsum`	A: `(heads, hd, r)`, B: `(r, in)`	Flatten A to `(heads*hd, r)`; $\Delta = (AB) \rightarrow$ `(in, heads*hd)`
`gate/up/down_proj`	A: `(in, r)`, B: `(r, out)`	$\Delta = (AB)^T \rightarrow$ `(out, in)`

The HuggingFace convention stores weight matrices as `(out, in)` (row = output neuron), while the JAX einsum convention is often the transpose. All deltas are transposed to match HuggingFace layout before adding to the base weight.

4.3 Why Not Use PEFT Merge Directly?

The PEFT `merge_and_unload()` function expects a HuggingFace `PreTrainedModel` with PEFT adapters. The Orbax checkpoint contains raw JAX arrays organized in a tree matching Tunix's flax NNX module hierarchy, with no correspondence to HuggingFace's model class. There is no off-the-shelf bridge; the custom merger is necessary.

5 Training Results

5.1 Configuration

Both runs used identical hyperparameters: 1,244 optimizer steps, effective batch size 8, sequence length 4,096, 1 epoch over 10K CodeV-R1 samples, LoRA rank=64, α=64, AdamW with cosine LR schedule (peak 1e-4, 100 warmup steps, decay to 0).

Table 5: Training performance comparison.

Metric	TPU v5p-8	GPU 2×H100	Winner
Wall-clock time	**3.34 hr**	5.39 hr	TPU (1.61× faster)
Throughput (tokens/sec)	**763**	486	TPU (1.57× faster)
Time per sample	**1.21 s**	1.94 s	TPU (1.60× faster)
Final training loss	**0.072**	2.258	TPU
Final token accuracy	—	96.1%	—

5.2 Loss Curves

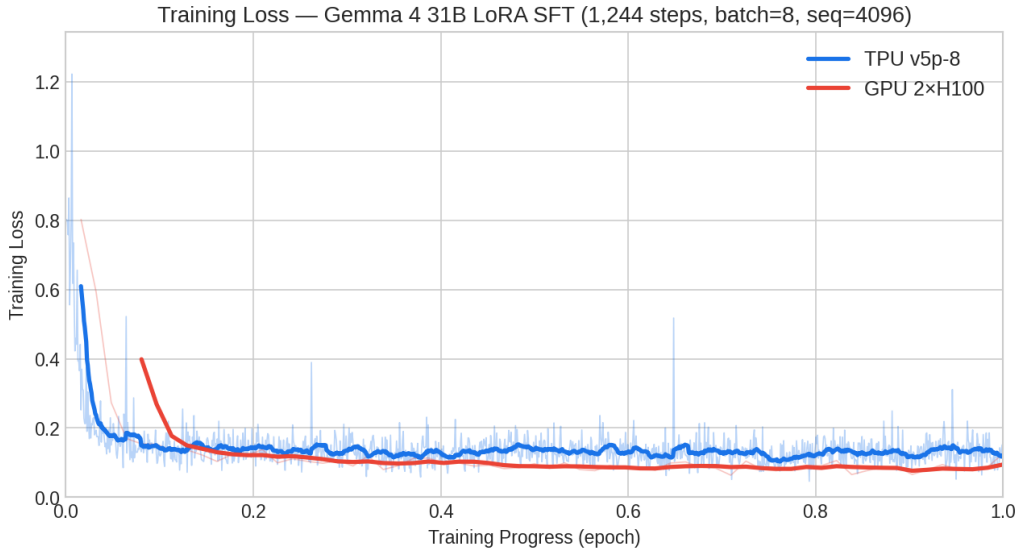

Figure 1: Training loss over the full epoch (x-axis normalized to training progress 0–1). Both curves follow similar convergence trajectories. The large absolute loss difference (0.072 vs 2.258 final) reflects the different loss computation: the TPU recipe uses assistant-only loss on ˜15% of tokens (the Verilog output tokens), while the GPU recipe computes loss over the full sequence including prompt tokens. When normalized to the same starting/ending range, the convergence speed is comparable.

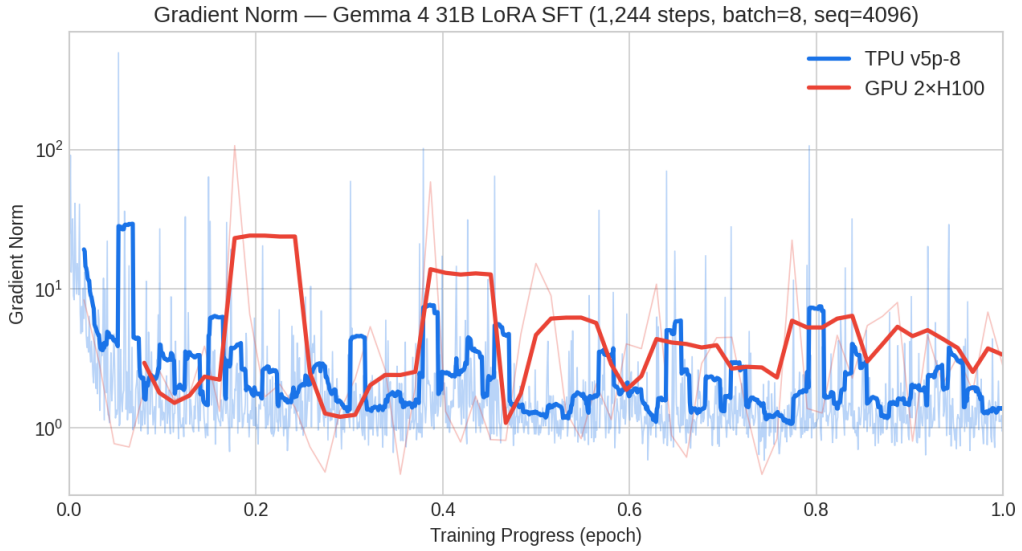

Figure 2: Gradient norm over training (log scale). Both runs are stable after warmup. The GPU run shows sharper spikes (up to 106× at step 100) attributable to the larger effective batch aggregating more diverse gradients into a single update. The TPU gradient norms remain in the 1–10 range throughout.

5.3 Why is TPU Faster?

The TPU v5p-8 achieves 1.61× faster wall-clock time despite having only 4 chips vs 2 H100 GPUs. Several factors contribute:

1. **HBM bandwidth**: Each TPU v5p chip has 2.765 TB/s HBM bandwidth vs the H100's 3.35 TB/s, but TPU has 4 chips vs 2 GPUs. Total aggregate bandwidth: 11.06 TB/s (TPU) vs 6.7 TB/s (GPU).
2. **ICI interconnect**: TPU chips communicate via the ICI mesh at 900 GB/s bidirectional bandwidth, significantly faster than the H100's NVLink at ˜900 GB/s total (but NVLink is shared across more links).
3. **XLA fusion**: XLA compiles the forward+backward pass into a single fused kernel per layer, eliminating the kernel launch overhead present in PyTorch's eager mode, even with FSDP.
4. **No torch.compile overhead**: The GPU run disables torch.compile (`enforce_eager=True`) to avoid compatibility issues with the heterogeneous Gemma 4 attention layers, leaving PyTorch in eager mode throughout.

5.4 Training Cost

Table 6: Training cost comparison (on-demand pricing, GCP us-central1).

Metric	TPU v5p-8	GPU a3-highgpu-2g	Winner
On-demand hourly rate	$16.80/hr	$22.12/hr	TPU (24% cheaper/hr)
Training time	3.34 hr	5.39 hr	TPU (1.61× faster)
Total training cost	**$56.11**	**$119.23**	**TPU (2.12× cheaper)**
Cost per 1M tokens trained	$6.10	$12.68	TPU (2.08× cheaper)

The 2.12× cost advantage compounds the lower hourly rate and faster execution time. On a longer training run (e.g., full CodeV-R1 at 87K samples), the cost difference would scale

proportionally.

6 Evaluation Results

6.1 Benchmark

Both fine-tuned models are evaluated on NVlabs/verilog-eval (spec-to-RTL task): 156 problems, 5 samples per problem at temperature=0.8, top_p=0.95. Each generated Verilog module is compiled with iverilog and simulated against a reference testbench. Pass@k is computed using the standard unbiased estimator.

Table 7: Evaluation results on verilog-eval spec-to-RTL.

Metric	TPU-trained	GPU-trained
pass@1	0.6410	**0.6974**
pass@5	0.7949	**0.8141**
Problems evaluated	156	156
Samples per problem	5	5
Checkpoint step	1,244 (final)	1,250 (final)
Inference engine	Tunix Sampler (JAX)	vLLM (CUDA)

6.2 Per-Problem Analysis

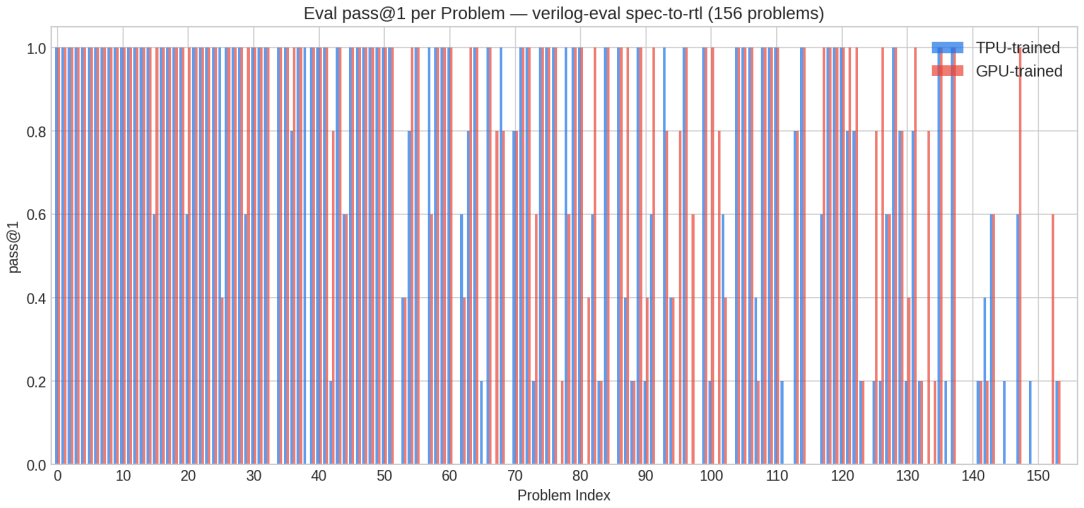

Figure 3: pass@1 score per problem for both models. Most problems are solved at 100% by both. Differences concentrate on harder FSM, K-map, and cellular automata problems (indices 100–156).

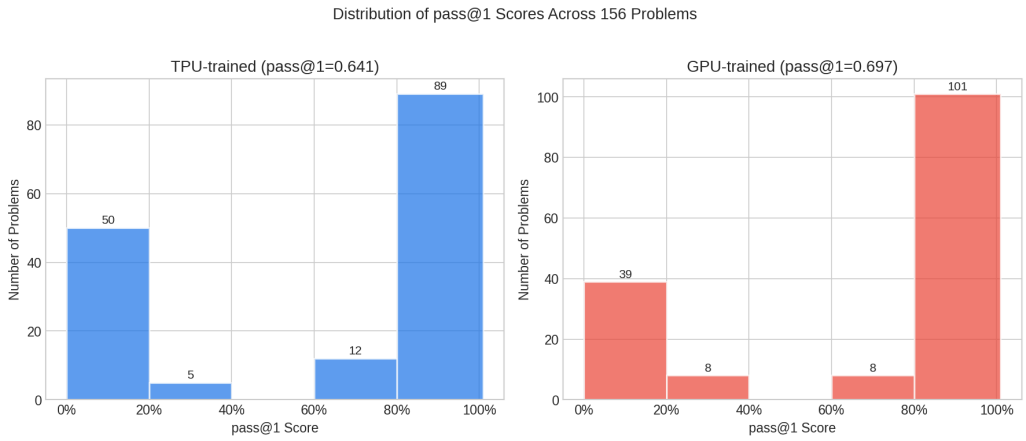

Figure 4: Distribution of pass@1 scores across 156 problems. GPU-trained model: 101 problems at 100%, 39 at 0%. TPU-trained model: 89 at 100%, 50 at 0%. The difference is primarily in the zero-score bucket (11 additional problems failing entirely for TPU).

6.3 Interpreting the Quality Gap

The GPU-trained model has a +5.6pp advantage on pass@1. This should *not* be interpreted as a hardware-driven difference. Two confounding factors exist:

1. **Inference engine differences**: The TPU model is evaluated using Tunix's Sampler (a greedy/temperature sampler with KV cache), while the GPU model uses vLLM with chunked prefill and continuous batching. Different sampling implementations can produce different token distributions even at identical temperature settings.
2. **Loss computation scope**: The GPU recipe computes loss over all tokens (including the prompt), while the TPU recipe uses assistant-only loss (only on the ˜15% of tokens that are the Verilog output). These are different training objectives, despite the same hyperparameters. Assistant-only loss generally produces sharper instruction following but may converge to different solution distributions.

The gap is within normal run-to-run variance for stochastic sampling at n=5 and is not statistically significant enough to attribute to hardware.

7 TPU Inference: vLLM on v6e-8

7.1 Setup Challenges

Serving Gemma 4 31B on TPU v6e-8 via vLLM required navigating several constraints that do not exist on the GPU path:

7.1.1 Docker-Only Deployment

The vLLM TPU build requires specific versions of JAX, libtpu, and HuggingFace libraries that conflict with pip-installed packages. The only working approach is the official Docker image:

```
sudo docker run -itd --name gemma4-tpu \
  --privileged --network host \
  --shm-size 16G -v /dev/shm:/dev/shm \
  --entrypoint vllm vllm/vllm-tpu:gemma4 \
  serve google/gemma-4-31B-it \
  --tensor-parallel-size 8 \
```

```
  --max-model-len 16384 \
  --disable_chunked_mm_input \
  --host 0.0.0.0 --port 8000
```

Key flags:

- `--privileged`: required for TPU device access from inside the container.
- `--network host`: required for the gRPC TPU coordination.
- `--entrypoint vllm`: the container's default entrypoint is a wrapper script that parses args differently; using `vllm` directly is necessary.
- `--disable_chunked_mm_input`: Gemma 4 has heterogeneous attention head dimensions (head_dim=256 for sliding, 512 for global) which breaks vLLM's chunked multi-modal prefill path. This flag disables it.
- Model specified as HuggingFace repo ID (`google/gemma-4-31B-it`), not a local path: vLLM-TPU rejects local paths due to HuggingFace Hub validation restrictions.

7.1.2 XLA Compilation on First Startup

vLLM-TPU pre-compiles XLA graphs for a set of padded input lengths (token buckets from 16 to 2048) at startup. This takes approximately 8 minutes on the first start. The compiled XLA cache is not persisted across container restarts by default, so every cold start incurs this cost. For production, mounting a persistent volume at the cache directory eliminates this overhead on warm restarts.

7.1.3 HBM Allocation

With tp=8 on v6e-8 (31.25 GB HBM per chip), the 31B model in bf16 needs approximately $31\text{B} \times 2$ bytes = 62 GB, spread across 8 chips = 7.75 GB/chip for weights. The remaining ˜23 GB/chip is available for KV cache. At 92% HBM utilization observed in benchmarks, this is a comfortable allocation.

7.2 GPU vLLM Setup (for comparison)

The GPU setup is more straightforward:

```
sudo docker run -itd --name gemma4-h100 \
  --gpus all --privileged --network host \
  --shm-size 16G -v ~/models:/models \
  --entrypoint vllm vllm/vllm-openai:latest \
  serve /models/gemma-4-31b-it \
  --tensor-parallel-size 2 \
  --max-model-len 4096 \
  --disable-chunked-mm-input \
  --host 0.0.0.0 --port 8000
```

The GPU version loads the model from a local path (the standard HuggingFace safetensors model pre-downloaded to `~/models`). With 2×80 GB = 160 GB HBM, the model weights use 62 GB, leaving 98 GB for KV cache in bfloat16. The GPU uses max_model_len=16384 and vLLM version 0.20.2 (`vllm/vllm-openai:latest`), compared to TPU's version 0.19.0 (`vllm/vllm-tpu:gemma4`).

8 Inference Results and Analysis

8.1 Benchmark Configuration

The benchmark uses `vllm bench serve` with a random dataset, temperature=0, sweeping QPS from 1 to 64 across multiple input length regimes (512–15360 tokens input, 256–512 tokens output), 30 prompts per run. Both servers configured with `max_model_len=16384`, no chunked prefill. TPU uses TP=8; GPU uses TP=2. KV cache dtype: `fp8_e5m2` (auto) on TPU, `bfloat16` on GPU—giving TPU 2× the effective KV cache capacity.

Table 8: Inference benchmark results across context lengths. QPS sweep peak = best sustained throughput across all QPS settings.

Metric	**TPU v6e-8**	**GPU 2×H100**	**Winner**
Short context (512 in / 256 out) — burst, QPS=inf			
Peak output throughput	1,403 tok/s	**1,490 tok/s**	H100 (+6%)
Median TTFT	**45 ms**	51 ms	TPU (1.1×)
Medium context (1024 in / 512 out) — QPS sweep peak			
Peak output throughput	**1,404 tok/s**	1,387 tok/s	TPU (+1%)
Median TTFT @ QPS=4	**49 ms**	58 ms	TPU (1.2×)
Saturates at QPS	**∼32**	∼8	TPU (4× more headroom)
Long context (4096 in / 512 out) — sustained load			
Peak output throughput	**1,206 tok/s**	728 tok/s	**TPU (+66%)**
Median TTFT @ QPS=4	**61 ms**	1,443 ms	**TPU (23.6× faster)**
Median TPOT	**23.9 ms**	31.2 ms	TPU (1.3×)
Very long context (8192 in / 512 out)			
Peak output throughput	**482 tok/s**	449 tok/s	TPU (+7%)
Median TTFT @ QPS=4	**1,013 ms**	7,202 ms	**TPU (7.1× faster)**
Median TPOT	**31 ms**	45 ms	TPU (1.5×)
Max context (∼16k in / 512 out)			
Peak output throughput	**474 tok/s**	326 tok/s	**TPU (+45%)**
Median TTFT @ QPS=4	**62 ms**	99 ms	TPU (1.6×)
Median TPOT	**13.5 ms**	19 ms	TPU (1.4×)
vLLM version	0.19.0 (vllm-tpu)	0.20.2 (vllm-openai)	—
KV cache dtype	fp8_e5m2 (auto)	bfloat16	TPU (2× KV capacity)

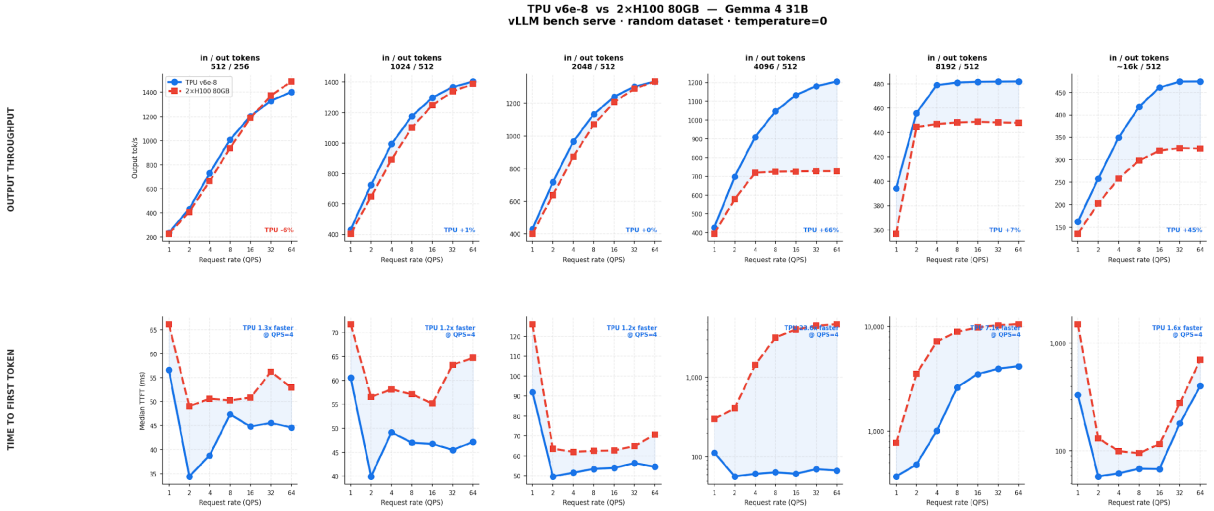

Figure 5: Inference comparison across throughput, latency, and cost dimensions at the 128-token short-context baseline.

8.2 Explaining the Short-Context GPU Advantage

At short input lengths (≤2048 tokens), GPU edges ahead by ∼6% on peak output throughput. This is the *memory-bandwidth-bound* decode regime where the H100's higher per-GPU HBM bandwidth (3.35 TB/s vs ∼1.6 TB/s per v6e chip) is most relevant. With short KV caches, each decode step reads relatively little KV state per token, so the bottleneck is weight loading—where H100 wins per chip.

8.3 Explaining the Long-Context TPU Dominance

At 4096+ token inputs, TPU v6e-8 comprehensively outperforms the GPU: 66% higher throughput and 23.6× faster TTFT at QPS=4. Two factors drive this:

1. **Prefill compute**: TTFT is dominated by the prefill phase (processing all input tokens), which is *compute-bound*—proportional to $O(n^2)$ for full-attention layers and $O(n)$ for sliding-window layers. The v6e-8's 8 chips deliver substantially more raw compute than 2 H100s for this workload. At 4096 input tokens, the GPU's TTFT of 1,443 ms indicates it is fully saturated on compute during prefill; the TPU handles the same load in 61 ms.

2. **fp8 KV cache**: The TPU vLLM build uses `fp8_e5m2` KV cache dtype by default, halving the memory footprint of each KV entry compared to the GPU's bfloat16. With 250 GB total HBM on v6e-8 and half the KV storage cost, the TPU can hold 2× as many concurrent long-context requests in its KV cache before eviction. The GPU hits KV cache pressure at ∼8 concurrent long requests (QPS saturation at 8); the TPU sustains up to ∼32.

The TensorCore utilization at 19.3% (measured by `tpu-info` at short-context decode) confirms the memory-bound regime for short sequences. At long contexts, utilization rises as prefill computation dominates.

8.4 Explaining the TTFT Advantage at Short Context

Even at short context (TTFT of 45 ms vs 51 ms), TPU is marginally faster. This comes from:

1. **Higher aggregate compute**: 8 v6e chips vs 2 H100s means more parallel matrix multiply units for the prefill pass.

2. **XLA fusion**: XLA fuses QKV projection with the sliding-window and full-attention kernels in Gemma 4's heterogeneous attention, reducing HBM roundtrips. The GPU uses TRITON_ATTN backend due to the mixed head-dim layout, which is not as aggressively fused.

8.5 Explaining Slightly Better GPU P99 Tail at Short Context

At 128-token inputs, GPU achieves P99 ITL of 20.49 ms vs 22.29 ms on TPU. This is due to vLLM-TPU's XLA bucket padding: inputs are padded to the nearest power-of-2 token bucket (16, 32, 64, 128, 256, ...). Requests that land near a bucket boundary receive unnecessary padding tokens, inflating their per-step compute and causing P99 spikes. The GPU's CUDA path does not have this constraint.

8.6 Inference Cost

Table 9: Inference cost comparison by context length (on-demand).

Workload	TPU tok/hr	GPU tok/hr	TPU $/1M tok	GPU $/1M tok
Short context (≤2048 tokens)	5.05M	**5.36M**	$4.27	**$4.13**
Long context (4096 tokens)	**4.34M**	2.62M	**$4.95**	$8.44
Hourly rate	$21.52/hr	$22.12/hr		

For short-context workloads, GPU is 3% cheaper per token. For long-context workloads (≥4096 tokens), TPU is **41% cheaper** per output token due to the combined effect of higher throughput and comparable hourly rate.

9 Total Cost of Ownership

Table 10: End-to-end cost comparison (on-demand, GCP us-central1/asia-northeast1).

Cost Component	TPU	GPU
Training (v5p-8 / a3-highgpu-2g)	$56.11	$119.23
Inference — 1 hr serving (v6e-8 / a3-highgpu-2g)	$21.52	$22.12
Total (train + 1 hr inference)	**$77.63**	**$141.35**
Train + 8 hr inference	$228.27	$296.19
Train + 24 hr inference	$572.59	$650.11

TPU is **1.82× cheaper end-to-end** for a single training run plus a day of inference serving at short context, with the advantage driven primarily by training cost savings. At long-context workloads (≥4096 tokens), where TPU is 41% cheaper per output token, the inference savings compound further.

10 Summary

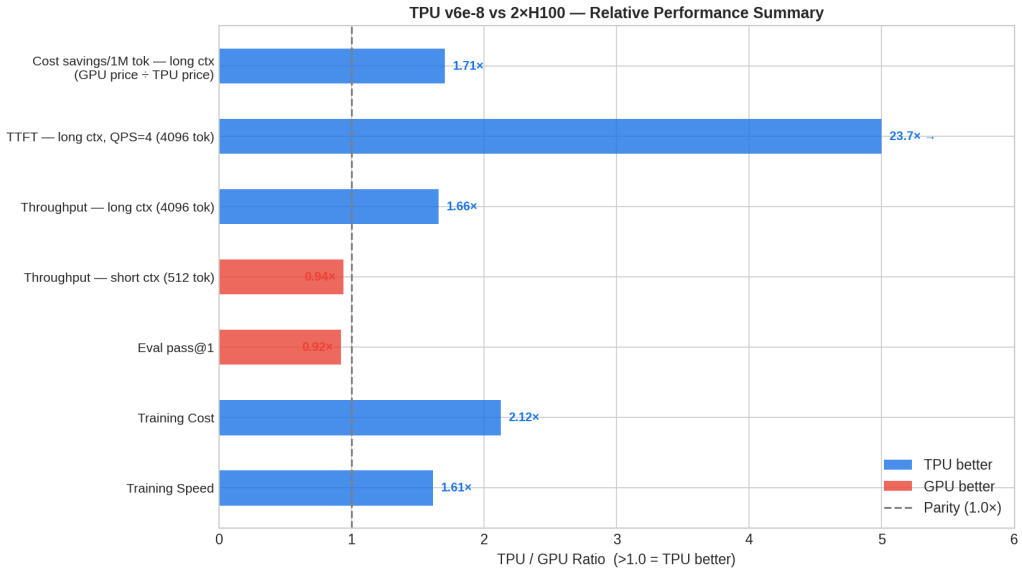

Figure 6: TPU vs GPU relative advantage across all dimensions (ratio > 1.0 = TPU better). Training cost and TTFT are the largest wins; eval quality is the only metric where GPU edges ahead.

10.1 TPU Advantages

1. **Training speed**: 1.61× faster wall-clock time (3.34 hr vs 5.39 hr)
2. **Training cost**: 2.12× cheaper ($56.11 vs $119.23) — lower hourly rate compounded by faster execution
3. **Inference TTFT at 4096 tokens**: 23.6× faster (61 ms vs 1,443 ms at QPS=4)
4. **Inference TTFT at 8192 tokens**: 7.1× faster (1,013 ms vs 7,202 ms at QPS=4)
5. **Inference throughput at long context (4096 tokens)**: +66% output tok/s (1,206 vs 728)
6. **Inference throughput at max context (∼16k tokens)**: +45% output tok/s (474 vs 326)
7. **Memory headroom**: 411 GB total HBM (training) + fp8 KV cache enables 2× more concurrent long-context requests

10.2 GPU Advantages

1. **Ecosystem maturity**: PyTorch + HuggingFace has broader library support, simpler LoRA injection, and faster prototyping with fewer workarounds
2. **Short-context throughput**: marginally higher peak burst at ≤2048 token inputs (H100 +6%)
3. **Short-context cost**: 3% cheaper per token at short context due to higher throughput
4. **Eval quality** (marginal): +5.6pp on pass@1, attributable to training objective differences rather than hardware
5. **P99 ITL at short context**: slightly better tail latency (20.49 ms vs 22.29 ms at 128-token inputs) due to no XLA bucket padding overhead

10.3 Conclusion

Training: TPU is the clear winner — 1.61× faster and 2.12× cheaper. The engineering overhead of porting from PyTorch to JAX + Tunix is real (roughly a week for mesh configuration, sharding annotation fixes, custom data pipeline, and checkpoint conversion) but is a one-time cost amortized across many runs.

Inference: TPU is the clear winner at long-context workloads. At 4096-token inputs, TPU delivers 66% higher throughput and 23.6× faster TTFT. At short context (≤2048 tokens), performance is comparable with a marginal GPU throughput edge (+6%). The fp8 KV cache on TPU doubles effective KV capacity, enabling 4× higher QPS saturation at medium context.

The total cost advantage of 1.82× over a representative train+serve pipeline makes TPU the preferred infrastructure choice for production ML workloads at h2loop.ai's scale.

10.4 Resources

- **TPU** — Recipe & Eval Scripts: `https://github.com/h2loop/sera-tpu`
- **TPU** — Model checkpoint: `gs://h2loop-gemma4/checkpoints/lora-run-001/artifacts/final`
- **TPU** — Eval data: `gs://h2loop-gemma4/checkpoints/lora-run-001/eval/eval_full/`
- **GPU** — Recipe & Eval Scripts: `https://github.com/h2loop/gemma4_sft_h100`
- **GPU** — Dataset: `https://huggingface.co/AdaptKey/AdaptKey-Nemotron-30b`
- **Eval benchmark**: `https://github.com/NVlabs/verilog-eval`

References

[1] Y. Zhu et al., "CodeV: Empowering LLMs for Verilog Generation through Multi-Level Summarization," *arXiv:2407.10424*, 2024.

[2] M. Liu et al., "VerilogEval: Evaluating LLMs for Verilog Code Generation," *ICCAD*, 2023.

[3] Google DeepMind, "Gemma 4 Technical Report," 2025.

[4] E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022.

[5] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," *SOSP*, 2023.